Selenium Installation and Troubleshooting Assignment

Python • WebDriver • Browser Automation • Diagnostic Skills

Names :
Pritha Dalui

Section : B

Roll No. : 87

Date :
27.08.2026

1. Assignment Purpose

Install and configure Selenium with Python, verify browser automation, investigate the underlying components, and diagnose common installation and execution failures. The emphasis is on independent technical problem-solving and evidence-based reporting.

4. Part A — Installation and Verification Tasks (20 marks)

1. Record the operating system, Python version, pip version, browser name, and browser version.

Ans: **Environment Record**

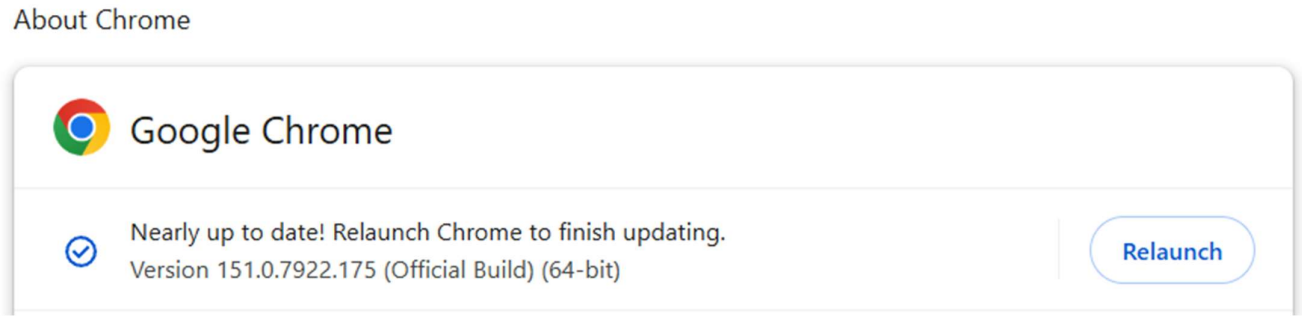
Component	Version
Operating System	Microsoft Windows 11 Home Single Language, OS Version 10.0.26200
Python	3.14.2
pip	25.3
Browser	Chrome
Browser version	151.0.7922.175

We recorded these versions before installing Selenium because Selenium depends on the Python environment and interacts with the installed browser. Recording the versions also helps diagnose compatibility problems later.

```
C:\Users\SAMADRITA HAZRA>python --version
Python 3.14.2

C:\Users\SAMADRITA HAZRA>pip --version
pip 25.3 from C:\Python314\Lib\site-packages\pip (python 3.14)

C:\Users\SAMADRITA HAZRA>systeminfo | findstr /B /C:"OS Name" /C:"OS Version"
OS Name:                   Microsoft Windows 11 Home Single Language
OS Version:                10.0.26200 N/A Build 26200
```



2. **Create a new project folder and a virtual environment. Activate it and show evidence that the environment is active.**

Ans: **Project folder:** selenium_assignment
Virtual environment: venv

A project folder named **selenium_assignment** was created and a Python virtual environment was set up inside it using **python -m venv venv**. The environment was activated on Windows Command Prompt using venv\Scripts\activate.bat. Successful activation was confirmed by two pieces of evidence: (1) the command prompt changed to display the (**venv**) prefix e.g. **(venv) C:\Users\SAMADRITA HAZRA\selenium_assignment>** and (2) running where python showed the virtual environment's Python interpreter **(...\selenium_assignment\venv\Scripts\python.exe)** listed first, confirming the isolated environment takes priority when python is executed.

```
C:\Users\SAMADRITA HAZRA>mkdir selenium_assignment

C:\Users\SAMADRITA HAZRA>cd selenium_assignment

C:\Users\SAMADRITA HAZRA\selenium_assignment>python -m venv venv

C:\Users\SAMADRITA HAZRA\selenium_assignment>dir
 Volume in drive C is OS
 Volume Serial Number is B8DF-E912

 Directory of C:\Users\SAMADRITA HAZRA\selenium_assignment

27-08-2026  23:31    <DIR>          .
27-08-2026  23:30    <DIR>          ..
27-08-2026  23:31    <DIR>          venv
               0 File(s)              0 bytes
               3 Dir(s)  20,182,720,512 bytes free

C:\Users\SAMADRITA HAZRA\selenium_assignment>venv\Scripts\activate.bat

(venv) C:\Users\SAMADRITA HAZRA\selenium_assignment>where python
C:\Users\SAMADRITA HAZRA\selenium_assignment\venv\Scripts\python.exe
C:\Python314\python.exe
C:\Users\SAMADRITA HAZRA\AppData\Local\Microsoft\WindowsApps\python.exe
C:\Users\SAMADRITA HAZRA\AppData\Local\Python\bin\python.exe

(venv) C:\Users\SAMADRITA HAZRA\selenium_assignment>
```

3. **Install Selenium using pip. Record the exact command and installed Selenium version.**

4. **Create a script that starts the browser, opens https://example.com, prints the page title and current URL, and closes the browser safely.**

5. **Run the script twice. Explain whether a driver was downloaded or discovered automatically and where you found the evidence.**

6. **Generate a requirements.txt file and explain how another student can reproduce the same environment.**

5. Part B — Technical Investigation Questions (30 marks)

Q1. What is Selenium, and which types of applications is it designed to automate? State two suitable and two unsuitable use cases.

Selenium is an open-source automation testing tool used to automate web browsers. It helps testers and developers perform tasks such as opening websites, clicking buttons, filling forms, and verifying web page elements automatically. Selenium mainly supports the automation and testing of **web applications** and works with browsers like Chrome, Firefox, Edge, and Safari.

Suitable use cases:

1. Automating the testing of web applications, such as login pages, registration forms, and online shopping websites.
2. Performing cross-browser testing to ensure a website works correctly on different browsers.

Unsuitable use cases:

1. Testing desktop applications like Notepad or Microsoft Word, as Selenium only supports web browsers.
2. Testing native mobile applications, which require tools like Appium instead of Selenium.

Conclusion:
Selenium is an effective tool for automating browser-based web applications, but it is not suitable for desktop software or native mobile app testing.

Q2. Differentiate Selenium WebDriver, Selenium IDE, and Selenium Grid.

Feature	Selenium WebDriver	Selenium IDE	Selenium Grid
Purpose	Automates browsers using programming code.	Records and plays back browser actions.	Runs tests on multiple browsers and machines simultaneously.
Coding Required	Yes	No	Yes
Flexibility	High	Limited	High
Best Use	Complex automation testing and frameworks.	Learning Selenium and creating simple test cases.	Parallel and cross-browser testing to reduce execution time.

- **Selenium WebDriver** is the main component of Selenium. It allows testers to write automation scripts in languages such as Python or Java and is suitable for advanced testing.

- **Selenium IDE** is a browser extension that records and replays user actions. It is useful for beginners and simple automation tasks without writing code.
- **Selenium Grid** allows WebDriver tests to run on multiple browsers, operating systems, or machines at the same time, making testing faster and more efficient.

Conclusion:
WebDriver is used to create automation scripts, Selenium IDE is used for simple record-and-playback testing, and Selenium Grid is used to execute tests across multiple environments in parallel.

Q3. Explain the roles of the test script, Selenium client library, browser driver, browser, and WebDriver protocol during one command such as driver.get().

When the command `driver.get()` is executed, different Selenium components work together to open a webpage.

- **Test Script:** It is the Python (or other language) code written by the tester. It contains the command `driver.get()` to open a website.
- **Selenium Client Library:** It receives the command from the test script and converts it into a format that the browser driver understands.
- **Browser Driver:** It acts as a bridge between Selenium and the browser. It receives the command and sends it to the browser.
- **Browser:** It performs the requested action, such as opening the specified webpage.
- **WebDriver Protocol:** It is the communication standard that allows the Selenium Client Library and the browser driver to exchange commands and responses.

In simple words: The test script sends the command, the Selenium Client Library translates it, the browser driver forwards it to the browser, the browser opens the webpage, and the result is sent back through the same process.

Q4. Why is a virtual environment recommended? What problem can occur when packages are installed globally?

A virtual environment is recommended because it creates a separate Python environment for each project. This allows every project to have its own packages and dependencies without affecting other projects.

If packages are installed globally, different projects may require different versions of the same package, which can lead to version conflicts. Updating a package for one project may also break another project. Using a virtual environment avoids these issues and keeps the project organized and easier to manage.

Q5. What is Selenium Manager? How has it changed driver setup in recent Selenium versions?

Selenium Manager is a built-in feature introduced in Selenium 4.6 that automatically manages browser drivers such as ChromeDriver and GeckoDriver.

Earlier, users had to download the correct browser driver manually, store it on their system, and specify its path in the code. With Selenium Manager, Selenium automatically detects the installed browser, downloads the compatible driver if needed, and configures it automatically.

This has made Selenium setup much easier by reducing manual configuration, preventing driver version mismatch errors, and allowing users to start browser automation with just a few lines of code.

Q6. Differentiate driver.find_element() and driver.find_elements(). What is returned when no element matches?

`driver.find_element()` and `driver.find_elements()` are both used to locate web elements, but they behave differently.

driver.find_element()	driver.find_elements()
Returns the **first matching element**.	Returns **all matching elements** as a list.
Used when only one element is expected.	Used when multiple elements may match the locator.
If no element is found, it raises a **NoSuchElementException**.	If no element is found, it returns an **empty list (`[]`)** without raising an exception.

Conclusion:
I would use `find_element()` when I need to interact with a single specific element, and `find_elements()` when I want to work with multiple matching elements or safely check whether any elements exist.

Q7. Compare ID, name, class name, CSS selector, and XPath locators. Which locator strategy would you prefer for stable tests, and why?

Locator	Description	Advantages	Limitations
ID	Finds an element using its unique ID.	Fast, reliable, and easy to use.	Works only if the ID is unique and stable.
Name	Finds an element using its `name` attribute.	Simple and readable.	May not be unique on the page.
Class Name	Finds an element using its CSS class.	Easy to use for common elements.	Multiple elements often share the same class.
CSS Selector	Finds elements using CSS rules.	Fast, flexible, and supports complex selectors.	Can become difficult to read if very complex.
XPath	Finds elements using their XML/HTML path.	Very powerful and can locate almost any element.	Longer, slower, and more likely to break if the page structure changes.

Preferred Locator Strategy:

For stable automation tests, I would prefer using the **ID locator** whenever it is available because it is unique, fast, and less likely to change. If an ID is not available, I would use a **CSS**

Selector because it is efficient and easier to maintain than XPath. I would use **XPath** only when other locators cannot uniquely identify the element.

Conclusion:
Choosing the right locator improves the reliability and maintainability of Selenium tests. In most cases, **ID** is the best choice, followed by **CSS Selector**, while **XPath** should be used only when necessary.

Q8. Differentiate implicit wait, explicit wait, and time.sleep(). Give one appropriate use case for each.

Implicit Wait	Explicit Wait	`time.sleep()`
Applies to all element searches in the script.	Waits for a specific condition before proceeding.	Pauses the execution for a fixed amount of time.
Set only once for the WebDriver.	Applied only where needed.	Does not check whether the element is ready.
More flexible than fixed delays.	Most efficient and reliable.	Can make tests slower if overused.

Use Cases:

- **Implicit Wait:** When most elements on a website take a few seconds to load.
- **Explicit Wait:** Waiting for a button to become clickable or an element to be visible before interacting with it.
- `time.sleep()`**:** Useful for short debugging or when waiting for a simple animation during testing.

Conclusion:
For reliable Selenium tests, **Explicit Wait** is the preferred choice because it waits only until the required condition is met. Implicit Wait can be used for general delays, while `time.sleep()` should be used only when necessary.

Q9. Why should driver.quit() normally be placed in a finally block or teardown method? How is it different from driver.close()?

`driver.quit()` should be placed in a `finally` **block** or **teardown method** so that the browser is closed even if an exception occurs during test execution. This prevents browser windows and driver processes from remaining open, which can waste system resources.

Difference between `driver.quit()` **and** `driver.close()`**:**

driver.quit()	driver.close()
Closes all browser windows and ends the WebDriver session.	Closes only the current browser window.
Stops the browser driver completely.	The WebDriver session continues if other windows are open.
Used at the end of the test execution.	Used when only one tab or window needs to be closed.

Conclusion:
I would normally use `driver.quit()` at the end of every test because it completely closes the browser and releases all resources, whereas `driver.close()` closes only the active window.

Q10. What causes StaleElementReferenceException? Describe two robust ways to handle it without hiding a genuine application defect.

A `StaleElementReferenceException` occurs when Selenium tries to interact with a web element that is no longer attached to the current webpage. This usually happens when the page is refreshed, reloaded, or the element is recreated dynamically after it was first located.

Two ways to handle it:

1. **Locate the element again before interacting with it.**
   Instead of reusing the old reference, find the element again after the page has changed.
2. **Use an Explicit Wait.**
   Wait until the element is visible or clickable before performing any action. This ensures Selenium interacts with the updated element.

Conclusion:
Instead of ignoring the exception, it is better to locate the element again or use explicit waits. These methods make the test more reliable while still allowing genuine application issues to be identified.

Q11. What is headless browser execution? Identify two benefits and two limitations.

Headless browser execution means running a web browser **without opening its graphical user interface (GUI)**. The browser performs all actions in the background, making it useful for automated testing.

Benefits:

1. **Faster execution:** Since the browser window is not displayed, tests generally run faster.
2. **Suitable for CI/CD:** It is ideal for automated testing on servers where no graphical interface is available.

Limitations:

1. **Difficult to debug:** Since the browser window is not visible, it is harder to identify UI-related issues.
2. **Rendering differences:** Some websites may behave slightly differently in headless mode compared to normal browser mode.

Conclusion:
Headless execution is useful for fast and automated testing, especially in CI/CD pipelines. However, for debugging or verifying UI behavior, running the browser in normal mode is often more effective.

Q12. Explain how browser and driver version mismatch can affect a Selenium session. Is this still relevant when Selenium Manager is used? Justify.

A browser and driver version mismatch occurs when the installed browser version is not compatible with the browser driver (such as ChromeDriver or GeckoDriver). In such cases, Selenium may fail to start the browser or throw session creation errors, preventing the automation script from running successfully.

With **Selenium Manager**, this problem is much less common because it automatically detects the installed browser version and downloads or uses the correct driver version. This reduces manual configuration and minimizes compatibility issues.

However, version mismatch can still occur in some situations, such as:

- Using an older version of Selenium that does not support Selenium Manager.
- Working in environments without internet access where Selenium Manager cannot download the required driver.
- Using custom or manually configured browser drivers.

Conclusion:
Browser and driver version mismatch can prevent Selenium tests from running correctly. Selenium Manager has greatly simplified driver management and reduced these issues, but it is still important to keep Selenium and the browser updated to avoid compatibility problems.

Q13. Why can a script work on one computer but fail on another? Discuss at least four environment-related reasons.

A Selenium script may work on one computer but fail on another because the environments are different. Some common reasons are:

1. **Different browser or browser driver versions:** An incompatible browser and driver version can cause Selenium to fail.
2. **Different Python or Selenium versions:** Using different versions may lead to compatibility issues.
3. **Missing packages or incorrect virtual environment:** Required libraries may not be installed, or the virtual environment may not be activated.
4. **Different operating system or system settings:** File paths, permissions, or browser configurations may vary between computers.

Conclusion:
To ensure consistent execution, all systems should have the same software versions, required packages, and proper environment configuration.

Q14. What information should be collected before reporting a Selenium defect so another person can reproduce it?

Before reporting a Selenium defect, the following information should be collected:

1. Clear steps to reproduce the issue.
2. Expected result and actual result.

3. Browser name, browser version, operating system, and Selenium version.
4. Error message or stack trace, if available.
5. Relevant screenshots or screen recordings.
6. Test script or code snippet related to the issue.

Conclusion:
Providing complete and accurate information helps others reproduce the issue easily and identify the root cause more quickly.

Q15. Identify three ethical or security precautions required when automating a website.

When automating a website, the following precautions should be followed:

1. **Do not use real user credentials or sensitive data** in automation scripts. Use test accounts and sample data whenever possible.
2. **Respect the website's policies and permissions.** Automate only websites or applications for which you have authorization.
3. **Avoid overloading the website** by sending too many automated requests, as this may affect system performance or violate usage policies.

Conclusion:
Following ethical and security practices helps protect user data, prevents misuse of automation, and ensures testing is performed responsibly.